

Разработка и верификация инструкций динамического разрешения в статически типизированной виртуальной машине.

М. А. Грошев^{1,2}

¹Московский физико-технический институт (национальный исследовательский университет)

² «Российский исследовательский институт»

В настоящее время большой популярностью пользуются языки со статической типизацией. Однако в условиях существования обширной кодовой базы, написанной на динамически типизированных языках программирования, на первый план выходит задача обеспечения их интероперабельности. Особенно актуальной становится поддержка межъязыкового взаимодействия на уровне виртуальной машины при сохранении безопасности выполнения и возможности статической верификации. Существующие платформы, такие как Android Runtime (ART) и Java Virtual Machine (JVM), предоставляют механизмы межъязыкового взаимодействия. Однако они не содержат специализированного набора инструкций, ориентированного непосредственно на такие сценарии. Разработка и верификация подобных инструкций являются целью данной работы.

Целевая платформа в своей системе типов поддерживает типы, разрешаемые динамически. К ним относятся union-тип — составной тип данных, позволяющий объекту во время выполнения принимать значение одного из заранее определённых типов, — а также any-тип, допускающий присвоение объекту значения произвольного типа, включая типы, определённые на динамической стороне.

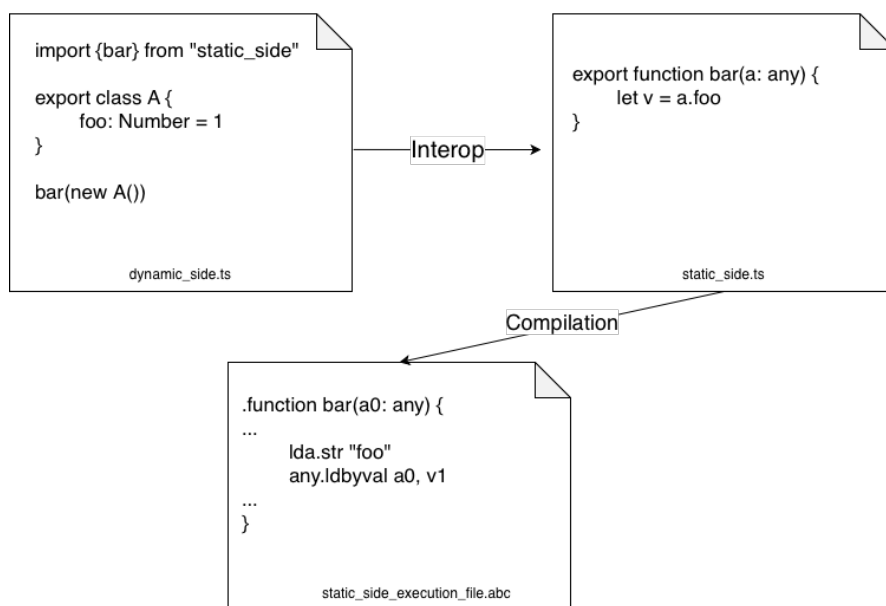


Рис. 1. Диаграмма, демонстрирующая использование any инструкции.

В рамках работы были исследованы и разработаны алгоритмы обработчиков any байт-кодов, с учетом основных требований к реализации: производительность, соответствие спецификации и повышение удобства миграции кодовой базы с динамически типизированного языка программирования.

Кроме того, в систему верификации типов были интегрированы union-типы и разработаны алгоритмы их статической проверки. Это позволило повысить безопасность платформы и предотвратить ряд ошибок времени выполнения ещё на этапе верификации при установке.

Таким образом, предложенные решения, значительно повышают интероперабельность языка, тем самым повышая скорость разработки при этом сохраняя возможности для верификации, что повышает безопасность исполнения.

Литература

1. ArkTs Documentation [Эл. ресурс]. URL: https://gitcode.com/igelhaus/arkcompiler_runtime_core/releases (дата обращения: 23.02.2026).
2. Ecma Documentation [Эл. ресурс]. URL: <https://tc39.es/ecma262/> (дата обращения: 23.02.2026)